Pollinator Classifier — Full Pipeline & Comparison

Trains both models sequentially, saves histories automatically, then generates all comparison visuals from real results.

Section	What happens
Part A	Shared setup — Drive, imports, datasets
Part B	Baseline CNN — build, train, evaluate, save
Part C	Improved EfficientNetV2S — build, train (2 phases), evaluate, save
Part D	Comparison visuals — all charts from real numbers

Part A — Shared Setup

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
print('Num GPUs:', len(tf.config.list_physical_devices('GPU')))
```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
Num GPUs: 1
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
import numpy as np
import pickle
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

print('TF version:', tf.__version__)
```

```
TF version: 2.20.0
```

```
# ── Config — edit base_path if your folder is named differently ────────────────
base_path  = '/content/drive/MyDrive/Colab Notebooks/pollinators/'
IMG_SIZE   = (224, 224)
AUTOTUNE   = tf.data.AUTOTUNE
SAVE_FIGS  = True

import os
if SAVE_FIGS:
    os.makedirs(base_path + 'comparison_figs/', exist_ok=True)
```

```
# ── Load raw datasets (shared by both models) ──────────────────────────────────
# Baseline uses batch=32, Improved uses batch=64 — we load both here
full_ds_32 = tf.keras.utils.image_dataset_from_directory(
    base_path + 'train',
    image_size=IMG_SIZE, batch_size=32, shuffle=True, seed=42
)
full_ds_64 = tf.keras.utils.image_dataset_from_directory(
    base_path + 'train',
    image_size=IMG_SIZE, batch_size=64, shuffle=True, seed=42
)
test_ds_raw_32 = tf.keras.utils.image_dataset_from_directory(
    base_path + 'test', image_size=IMG_SIZE, batch_size=32
)
test_ds_raw_64 = tf.keras.utils.image_dataset_from_directory(
    base_path + 'test', image_size=IMG_SIZE, batch_size=64
)

class_names = full_ds_32.class_names
```

```
num_classes  = len(class_names)
print(f'Classes ({num_classes}): {class_names}')

# 80/20 train/val splits
def split_ds(full_ds):
    total  = len(full_ds)
    n_train = int(0.8 * total)
    return full_ds.take(n_train), full_ds.skip(n_train), n_train, total - n_train

train_raw_32, val_raw_32, tr32, vl32 = split_ds(full_ds_32)
train_raw_64, val_raw_64, tr64, vl64 = split_ds(full_ds_64)
print(f'Batch=32 → train: {tr32}  val: {vl32}')
print(f'Batch=64 → train: {tr64}  val: {vl64}')
```

```
Found 12594 files belonging to 100 classes.
Found 12594 files belonging to 100 classes.
Found 500 files belonging to 100 classes.
Found 500 files belonging to 100 classes.
Classes (100): ['ADONIS', 'AFRICAN GIANT SWALLOWTAIL', 'AMERICAN SNOOT', 'AN 88', 'APPOLLO', 'ARCIGERA FLOWER MOTH',
Batch=32 → train: 315  val: 79
Batch=64 → train: 157  val: 40
```

Part B — Baseline CNN (from scratch)

```
# ── B1. Normalization pipeline (baseline rescales to [0,1]) ──────────────────
norm = layers.Rescaling(1./255)

train_ds_a = train_raw_32.map(lambda x, y: (norm(x), y)).cache().shuffle(1000).prefetch(AUTOTUNE)
val_ds_a   = val_raw_32.map(lambda x, y: (norm(x), y)).cache().prefetch(AUTOTUNE)
test_ds_a  = test_ds_raw_32.map(lambda x, y: (norm(x), y)).cache().prefetch(AUTOTUNE)
```

```
# ── B2. Data augmentation ────────────────────────────────────────────────────
aug_a = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomFlip('vertical'),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomContrast(0.2),
], name='augmentation_baseline')
```

```
# ── B3. Build model ──────────────────────────────────────────────────────────
model_a = models.Sequential([
    aug_a,
    # Block 1
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # Block 2
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # Block 3
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # Block 4
    layers.Conv2D(256, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # Block 5
    layers.Conv2D(512, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu',
                 kernel_regularizer=tf.keras.regularizers.l2(0.005)),
    layers.Dropout(0.6),
    layers.Dense(num_classes, activation='softmax')
])
```

```
model_a.build(input_shape=(None, 224, 224, 3))
model_a.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
augmentation_baseline (Sequential)	(None, 224, 224, 3)	0
conv2d (Conv2D)	(None, 224, 224, 32)	896
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
conv2d_3 (Conv2D)	(None, 28, 28, 256)	295,168
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)	0
conv2d_4 (Conv2D)	(None, 14, 14, 512)	1,180,160
batch_normalization_4 (BatchNormalization)	(None, 14, 14, 512)	2,048
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 100)	25,700

Total params: 1,729,572 (6.60 MB)
Trainable params: 1,727,588 (6.59 MB)
Non-trainable params: 1,984 (7.75 KB)

```
# ── B4. Compile ──────────────────────────────────────────────────────────
model_a.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

callbacks_a = [
    EarlyStopping(monitor='val_accuracy', patience=7,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                      patience=3, min_lr=1e-7, verbose=1)
]
```

```
# ── B5. Train ────────────────────────────────────────────────────────────
print('Training Baseline CNN...')
history_a = model_a.fit(
    train_ds_a,
    validation_data=val_ds_a,
    epochs=50,
    callbacks=callbacks_a
)
```

```
Training Baseline CNN...
Epoch 1/50
```

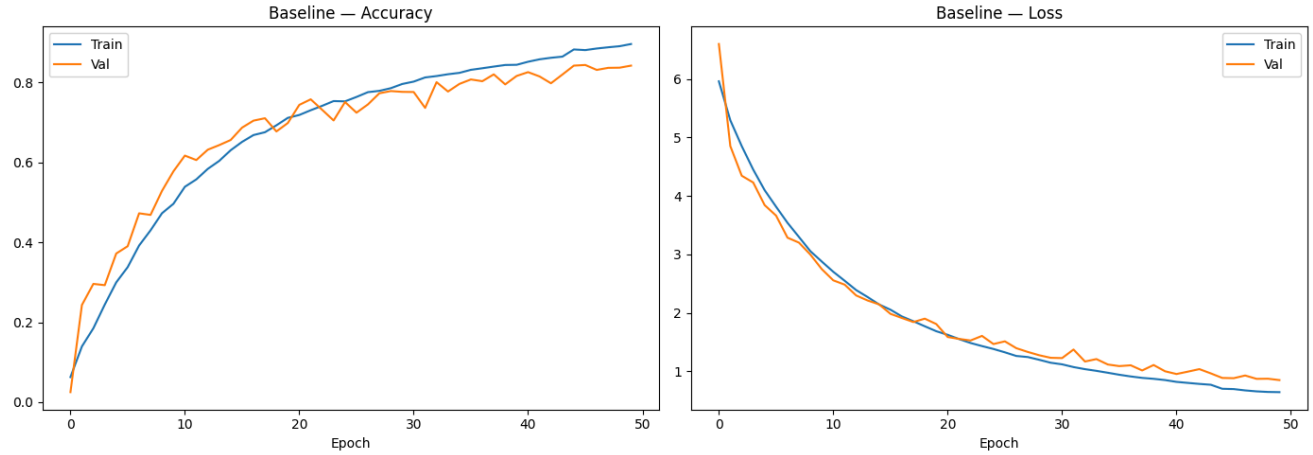
```
315/315 ━━━━━━━━━━━━━━━━━━━━ 753s 540ms/step - accuracy: 0.0627 - loss: 5.9579 - val_accuracy: 0.0251 - val_loss: 6.
Epoch 2/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.1394 - loss: 5.2922 - val_accuracy: 0.2430 - val_loss: 4.85
Epoch 3/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.1848 - loss: 4.8502 - val_accuracy: 0.2959 - val_loss: 4.34
Epoch 4/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.2442 - loss: 4.4501 - val_accuracy: 0.2928 - val_loss: 4.22
Epoch 5/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.2997 - loss: 4.0979 - val_accuracy: 0.3719 - val_loss: 3.84
Epoch 6/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.3378 - loss: 3.8153 - val_accuracy: 0.3902 - val_loss: 3.60
Epoch 7/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.3920 - loss: 3.5398 - val_accuracy: 0.4726 - val_loss: 3.28
Epoch 8/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.4300 - loss: 3.2978 - val_accuracy: 0.4686 - val_loss: 3.20
Epoch 9/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.4727 - loss: 3.0545 - val_accuracy: 0.5286 - val_loss: 2.99
Epoch 10/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.4963 - loss: 2.8770 - val_accuracy: 0.5776 - val_loss: 2.74
Epoch 11/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.5389 - loss: 2.7031 - val_accuracy: 0.6169 - val_loss: 2.55
Epoch 12/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.5574 - loss: 2.5491 - val_accuracy: 0.6058 - val_loss: 2.48
Epoch 13/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.5836 - loss: 2.3905 - val_accuracy: 0.6317 - val_loss: 2.29
Epoch 14/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6037 - loss: 2.2717 - val_accuracy: 0.6432 - val_loss: 2.21
Epoch 15/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6303 - loss: 2.1450 - val_accuracy: 0.6559 - val_loss: 2.14
Epoch 16/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6513 - loss: 2.0548 - val_accuracy: 0.6870 - val_loss: 1.98
Epoch 17/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6684 - loss: 1.9395 - val_accuracy: 0.7045 - val_loss: 1.91
Epoch 18/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6754 - loss: 1.8578 - val_accuracy: 0.7104 - val_loss: 1.84
Epoch 19/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.6928 - loss: 1.7718 - val_accuracy: 0.6774 - val_loss: 1.90
Epoch 20/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7114 - loss: 1.6874 - val_accuracy: 0.6985 - val_loss: 1.81
Epoch 21/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7186 - loss: 1.6256 - val_accuracy: 0.7438 - val_loss: 1.58
Epoch 22/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7305 - loss: 1.5528 - val_accuracy: 0.7578 - val_loss: 1.55
Epoch 23/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7415 - loss: 1.4861 - val_accuracy: 0.7315 - val_loss: 1.57
Epoch 24/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7534 - loss: 1.4333 - val_accuracy: 0.7049 - val_loss: 1.60
Epoch 25/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7527 - loss: 1.3839 - val_accuracy: 0.7510 - val_loss: 1.40
Epoch 26/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7637 - loss: 1.3266 - val_accuracy: 0.7243 - val_loss: 1.51
Epoch 27/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7757 - loss: 1.2643 - val_accuracy: 0.7450 - val_loss: 1.39
Epoch 28/50
315/315 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7790 - loss: 1.2463 - val_accuracy: 0.7729 - val_loss: 1.33
```

```
# ── B6. Evaluate + Save ──────────────────────────────────────────────────
print('\n=== BASELINE — TEST SET ===')
loss_a, acc_a = model_a.evaluate(test_ds_a)
print(f'Test accuracy: {acc_a:.4f}  |  Test loss: {loss_a:.4f}')

model_a.save(base_path + 'pollinator_model.keras')
pickle.dump(history_a.history,
            open(base_path + 'history_baseline.pkl', 'wb'))
print('Model + history saved.')
```

```
=== BASELINE — TEST SET ===
16/16 ━━━━━━━━━━━━━━━━━━━━ 31s 2s/step - accuracy: 0.8580 - loss: 0.7481
Test accuracy: 0.8580  |  Test loss: 0.7481
Model + history saved.
```

```
# ── B7. Quick training plot ──────────────────────────────────────────────
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(history_a.history['accuracy'],     label='Train')
ax1.plot(history_a.history['val_accuracy'], label='Val')
ax1.set_title('Baseline — Accuracy'); ax1.set_xlabel('Epoch'); ax1.legend()
ax2.plot(history_a.history['loss'],     label='Train')
ax2.plot(history_a.history['val_loss'], label='Val')
ax2.set_title('Baseline — Loss'); ax2.set_xlabel('Epoch'); ax2.legend()
plt.tight_layout()
```

```
plt.savefig(base_path + 'training_history_baseline.png', dpi=150)
plt.show()
```



Part C — Improved Model (EfficientNetV2S + Transfer Learning)

```
# ── C1. Pipeline — EfficientNet expects raw [0,255] ──────────────────────────
train_ds_b = train_raw_64.cache().shuffle(1000).prefetch(AUTOTUNE)
val_ds_b   = val_raw_64.cache().prefetch(AUTOTUNE)
test_ds_b  = test_ds_raw_64.cache().prefetch(AUTOTUNE)
```

```
# ── C2. Augmentation (+ RandomBrightness vs baseline) ────────────────────────
aug_b = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomFlip('vertical'),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomContrast(0.2),
    layers.RandomBrightness(0.2),   # CHANGED: added
], name='augmentation_improved')
```

```
# ── C3. Build model — EfficientNetV2S backbone (frozen for phase 1) ──────────
base_model = tf.keras.applications.EfficientNetV2S(
    include_top=False, weights='imagenet', input_shape=(224, 224, 3)
)
base_model.trainable = False

inputs  = tf.keras.Input(shape=(224, 224, 3))
x       = aug_b(inputs)
x       = base_model(x, training=False)
x       = layers.GlobalAveragePooling2D()(x)
x       = layers.Dense(256, activation='relu',
              kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
x       = layers.Dropout(0.4)(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)

model_b = tf.keras.Model(inputs, outputs)
model_b.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/efficientnet_v2/efficientnetv2-s_n
82420632/82420632 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
Model: "functional_3"
```

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 224, 224, 3)	0
augmentation_improved (Sequential)	(None, 224, 224, 3)	0
efficientnetv2-s (Functional)	(None, 7, 7, 1280)	20,331,360
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 100)	25,700

```
Total params: 20,684,996 (78.91 MB)
Trainable params: 353,636 (1.35 MB)
Non-trainable params: 20,331,360 (77.56 MB)
```

```
# ── C4. Phase 1 — train head only (backbone frozen) ──────────────────────
model_b.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

callbacks_b = [
    EarlyStopping(monitor='val_accuracy', patience=7,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                      patience=3, min_lr=1e-7, verbose=1)
]

print('Phase 1 — training head only...')
history_phase1 = model_b.fit(
    train_ds_b, validation_data=val_ds_b,
    epochs=10, callbacks=callbacks_b
)
```

```
Phase 1 — training head only...
Epoch 1/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 66s 186ms/step - accuracy: 0.3216 - loss: 3.3209 - val_accuracy: 0.5841 - val_loss: 2.09
Epoch 2/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 65ms/step - accuracy: 0.6064 - loss: 1.9137 - val_accuracy: 0.7518 - val_loss: 1.492
Epoch 3/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.6964 - loss: 1.5720 - val_accuracy: 0.7942 - val_loss: 1.290
Epoch 4/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7362 - loss: 1.3985 - val_accuracy: 0.8162 - val_loss: 1.160
Epoch 5/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7572 - loss: 1.3005 - val_accuracy: 0.8339 - val_loss: 1.081
Epoch 6/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7770 - loss: 1.2114 - val_accuracy: 0.8370 - val_loss: 1.060
Epoch 7/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7733 - loss: 1.1767 - val_accuracy: 0.8504 - val_loss: 0.994
Epoch 8/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 62ms/step - accuracy: 0.7864 - loss: 1.1365 - val_accuracy: 0.8441 - val_loss: 0.975
Epoch 9/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7890 - loss: 1.1159 - val_accuracy: 0.8543 - val_loss: 0.927
Epoch 10/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 63ms/step - accuracy: 0.7984 - loss: 1.0801 - val_accuracy: 0.8692 - val_loss: 0.903
Restoring model weights from the end of the best epoch: 10.
```

```
# ── C5. Phase 2 — unfreeze top 50 layers and fine-tune ──────────────────
base_model.trainable = True
for layer in base_model.layers[:-50]:
    layer.trainable = False

print(f'Trainable layers: {sum(1 for l in model_b.layers if l.trainable)}')

model_b.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
```

```
)

print('Phase 2 — fine-tuning top 50 layers...')
history_phase2 = model_b.fit(
    train_ds_b, validation_data=val_ds_b,
    epochs=50, callbacks=callbacks_b
)
```

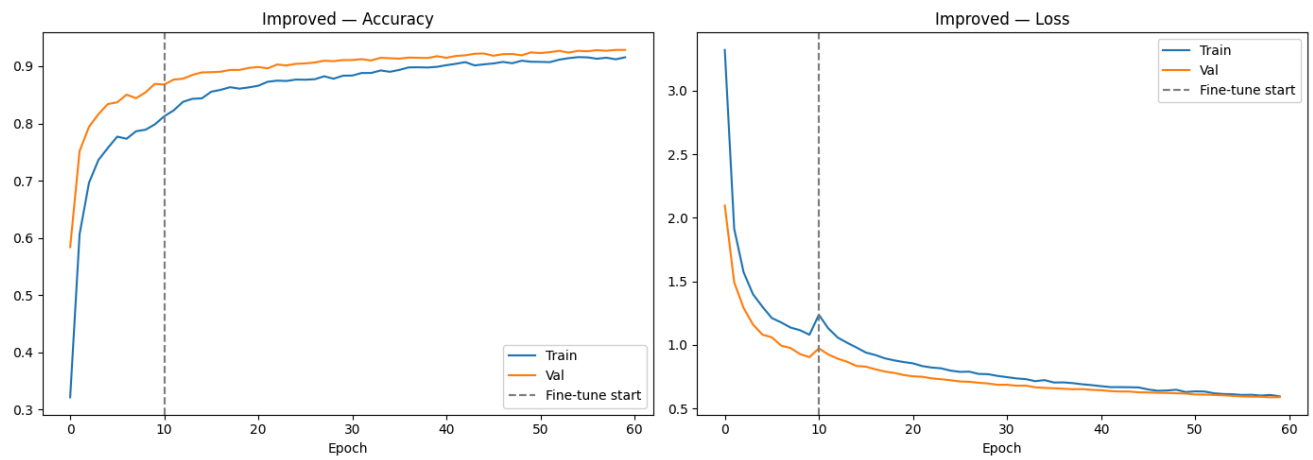
```
Epoch 23/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.8882 - loss: 0.7311 - val_accuracy: 0.9101 - val_loss: 0.6
Epoch 24/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 75ms/step - accuracy: 0.8926 - loss: 0.7149 - val_accuracy: 0.9148 - val_loss: 0.6
Epoch 25/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.8903 - loss: 0.7231 - val_accuracy: 0.9140 - val_loss: 0.6
Epoch 26/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 75ms/step - accuracy: 0.8936 - loss: 0.7042 - val_accuracy: 0.9132 - val_loss: 0.6
Epoch 27/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.8980 - loss: 0.7052 - val_accuracy: 0.9152 - val_loss: 0.6
Epoch 28/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.8982 - loss: 0.6995 - val_accuracy: 0.9148 - val_loss: 0.6
Epoch 29/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.8978 - loss: 0.6904 - val_accuracy: 0.9144 - val_loss: 0.6
Epoch 30/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.8990 - loss: 0.6839 - val_accuracy: 0.9175 - val_loss: 0.6
Epoch 31/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9019 - loss: 0.6755 - val_accuracy: 0.9148 - val_loss: 0.6
Epoch 32/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9043 - loss: 0.6681 - val_accuracy: 0.9179 - val_loss: 0.6
Epoch 33/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9071 - loss: 0.6680 - val_accuracy: 0.9191 - val_loss: 0.6
Epoch 34/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9015 - loss: 0.6671 - val_accuracy: 0.9218 - val_loss: 0.6
Epoch 35/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9034 - loss: 0.6655 - val_accuracy: 0.9222 - val_loss: 0.6
Epoch 36/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9049 - loss: 0.6498 - val_accuracy: 0.9183 - val_loss: 0.6
Epoch 37/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9075 - loss: 0.6396 - val_accuracy: 0.9211 - val_loss: 0.6
Epoch 38/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9053 - loss: 0.6414 - val_accuracy: 0.9214 - val_loss: 0.6
Epoch 39/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9096 - loss: 0.6472 - val_accuracy: 0.9191 - val_loss: 0.6
Epoch 40/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 75ms/step - accuracy: 0.9078 - loss: 0.6294 - val_accuracy: 0.9242 - val_loss: 0.6
Epoch 41/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9076 - loss: 0.6345 - val_accuracy: 0.9230 - val_loss: 0.6
Epoch 42/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9071 - loss: 0.6335 - val_accuracy: 0.9246 - val_loss: 0.6
Epoch 43/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9115 - loss: 0.6196 - val_accuracy: 0.9269 - val_loss: 0.6
Epoch 44/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9141 - loss: 0.6138 - val_accuracy: 0.9238 - val_loss: 0.6
Epoch 45/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9159 - loss: 0.6121 - val_accuracy: 0.9269 - val_loss: 0.5
Epoch 46/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9155 - loss: 0.6063 - val_accuracy: 0.9262 - val_loss: 0.5
Epoch 47/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9131 - loss: 0.6080 - val_accuracy: 0.9281 - val_loss: 0.5
Epoch 48/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9149 - loss: 0.6017 - val_accuracy: 0.9269 - val_loss: 0.5
Epoch 49/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.9121 - loss: 0.6063 - val_accuracy: 0.9285 - val_loss: 0.5
Epoch 50/50
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 74ms/step - accuracy: 0.9156 - loss: 0.5954 - val_accuracy: 0.9285 - val_loss: 0.5
Restoring model weights from the end of the best epoch: 49.
```

```
# ── C6. Evaluate + Save ──────────────────────────────────────────────────
print('\n=== IMPROVED — TEST SET ===')
loss_b, acc_b = model_b.evaluate(test_ds_b)
print(f'Test accuracy: {acc_b:.4f}  |  Test loss: {loss_b:.4f}')

# Combine both phase histories into one dict and save
phase1_len = len(history_phase1.history['accuracy'])
hist_b_combined = {
    'accuracy':     history_phase1.history['accuracy']     + history_phase2.history['accuracy'],
    'val_accuracy': history_phase1.history['val_accuracy'] + history_phase2.history['val_accuracy'],
    'loss':         history_phase1.history['loss']         + history_phase2.history['loss'],
    'val_loss':     history_phase1.history['val_loss']     + history_phase2.history['val_loss'],
    'phase1_epochs': phase1_len
}
```

```
model_b.save(base_path + 'pollinator_model_v2.keras')
pickle.dump(hist_b_combined,
            open(base_path + 'history_improved.pkl', 'wb'))
print('Model + history saved.')
```

```
=== IMPROVED — TEST SET ===
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 134ms/step - accuracy: 0.9340 - loss: 0.5328
Test accuracy: 0.9340  |  Test loss: 0.5328
Model + history saved.
```

```
# ── C7. Quick training plot (both phases) ──────────────────────────────
acc      = hist_b_combined['accuracy']
val_acc  = hist_b_combined['val_accuracy']
loss_c   = hist_b_combined['loss']
val_loss = hist_b_combined['val_loss']

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(acc, label='Train'); ax1.plot(val_acc, label='Val')
ax1.axvline(phase1_len, color='gray', ls='--', label='Fine-tune start')
ax1.set_title('Improved — Accuracy'); ax1.set_xlabel('Epoch'); ax1.legend()
ax2.plot(loss_c, label='Train'); ax2.plot(val_loss, label='Val')
ax2.axvline(phase1_len, color='gray', ls='--', label='Fine-tune start')
ax2.set_title('Improved — Loss'); ax2.set_xlabel('Epoch'); ax2.legend()
plt.tight_layout()
plt.savefig(base_path + 'training_history_improved.png', dpi=150)
plt.show()
```



Part D — Comparison Visuals

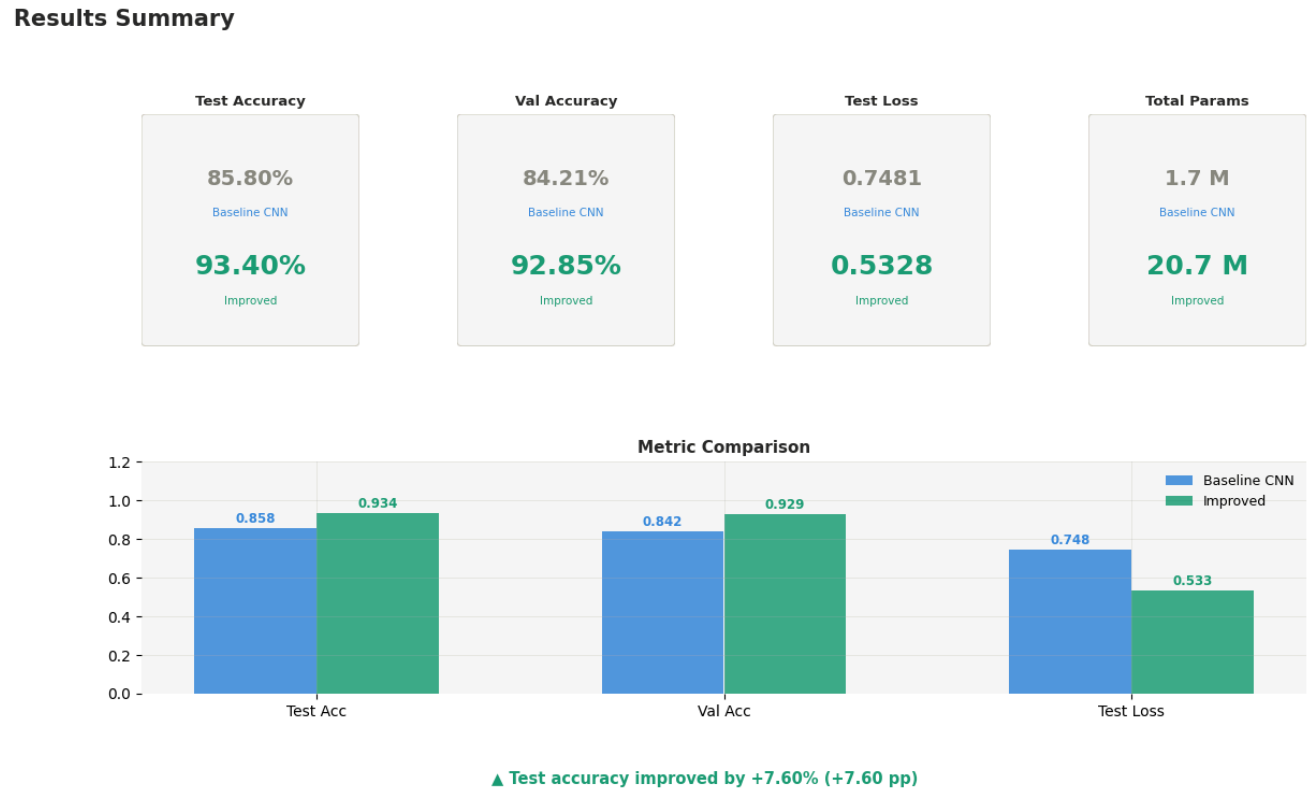
All charts are generated from the real training and test results above.

```
# ── D0. Style ──────────────────────────────────────────────────────────
BLUE   = '#378ADD'
TEAL   = '#1D9E75'
BLUE_L = '#B5D4F4'
TEAL_L = '#9FE1CB'
BG     = '#F8F8F6'
DARK   = '#2C2C2A'
GRAY   = '#888780'
LGRAY  = '#D3D1C7'

plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'axes.spines.top': False, 'axes.spines.right': False,
    'axes.spines.left': False, 'axes.spines.bottom': False,
    'axes.grid': True, 'grid.alpha': 0.35, 'grid.linewidth': 0.5,
    'axes.facecolor': BG, 'figure.facecolor': 'white',
})
```

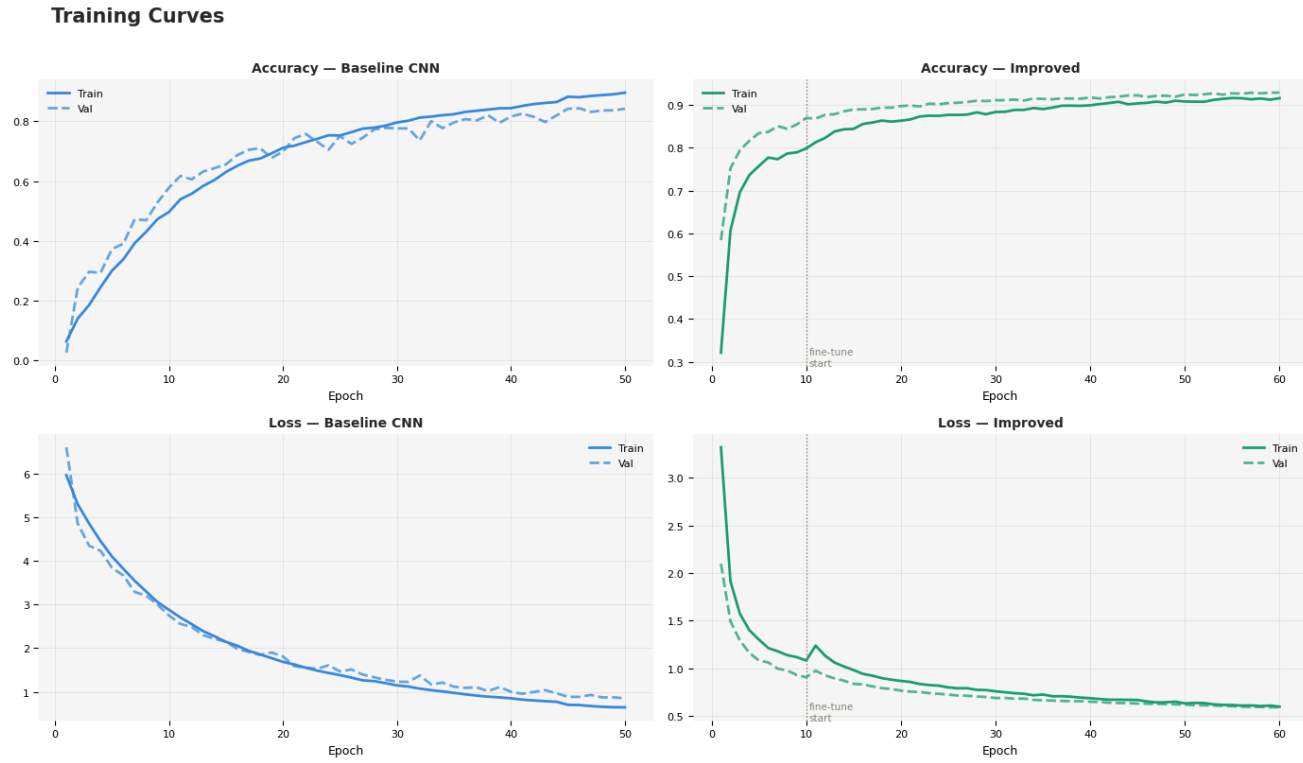
```
def savefig(name):
    if SAVE_FIGS:
        plt.savefig(base_path + 'comparison_figs/' + name, dpi=150, bbox_inches='tight')

# Pull val_accuracy from histories
val_acc_a = history_a.history['val_accuracy'][-1]
val_acc_b = hist_b_combined['val_accuracy'][-1]

params_a = model_a.count_params() / 1e6
params_b = model_b.count_params() / 1e6

print(f'Baseline  — test acc: {acc_a:.4f}  val acc: {val_acc_a:.4f}  params: {params_a:.1f}M')
print(f'Improved  — test acc: {acc_b:.4f}  val acc: {val_acc_b:.4f}  params: {params_b:.1f}M')
print(f'Delta     — {acc_b - acc_a:+.4f} ({(acc_b-acc_a)*100:+.2f} pp)')
```

```
Baseline  — test acc: 0.8580  val acc: 0.8421  params: 1.7M
Improved  — test acc: 0.9340  val acc: 0.9285  params: 20.7M
Delta     — +0.0760 (+7.60 pp)
```

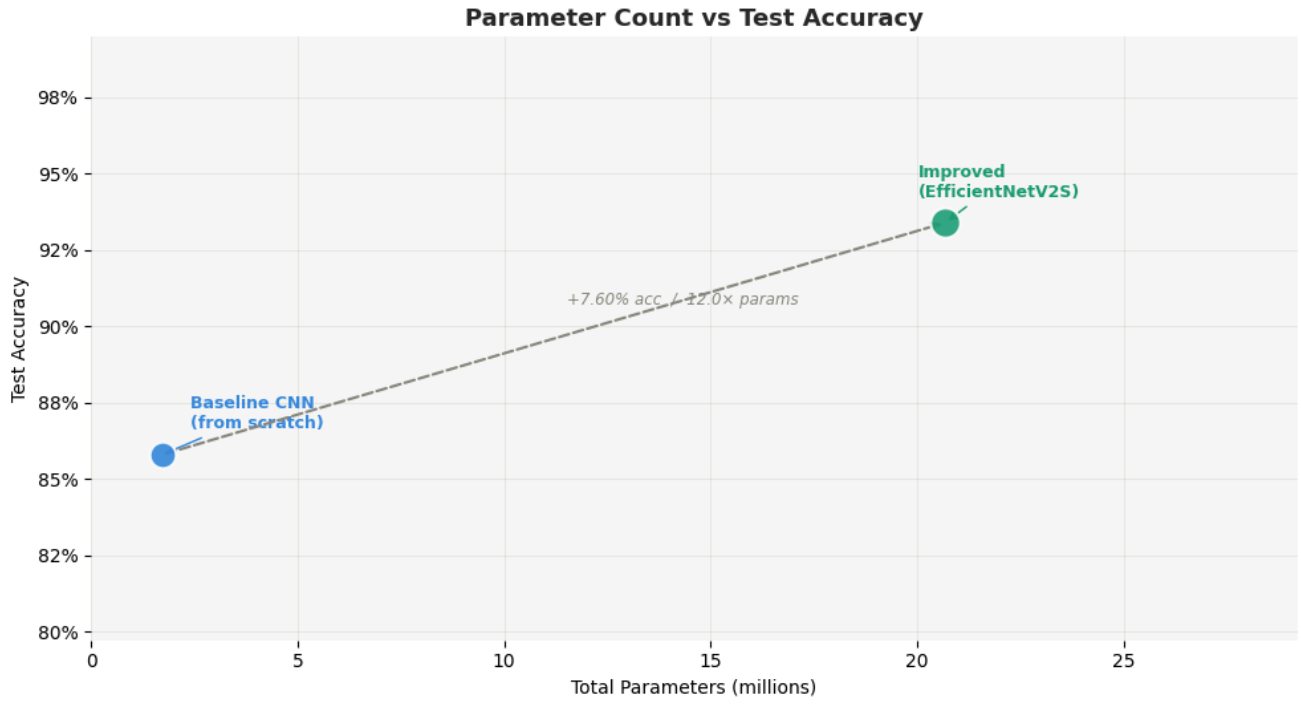
```
# ── D1. Results Dashboard ──────────────────────────────────────────────────
fig = plt.figure(figsize=(14, 7))
gs  = gridspec.GridSpec(2, 4, figure=fig, hspace=0.5, wspace=0.45)

card_data = [
    ('Test Accuracy', [acc_a,      acc_b],      '{:.2%}', 'higher'),
    ('Val Accuracy',  [val_acc_a,  val_acc_b],  '{:.2%}', 'higher'),
    ('Test Loss',     [loss_a,     loss_b],     '{:.4f}', 'lower'),
    ('Total Params',  [params_a,   params_b],   '{:.1f} M', 'higher'),
]
model_names = ['Baseline CNN', 'Improved']
colors_m    = [BLUE, TEAL]

for j, (label, vals, fmt, better) in enumerate(card_data):
    ax = fig.add_subplot(gs[0, j])
    ax.axis('off')
    best = max(vals) if better == 'higher' else min(vals)
    ax.add_patch(mpatches.FancyBboxPatch((0.02,0.02), 0.96, 0.96,
        boxstyle='round,pad=0.02', fc=BG, ec=LGRAY, lw=0.8,
        transform=ax.transAxes))
    for k, (m, v, c) in enumerate(zip(model_names, vals, colors_m)):
        yoff = 0.72 - k*0.38
        ax.text(0.5, yoff, fmt.format(v), transform=ax.transAxes,
                ha='center', va='center',
                fontsize=18 if v==best else 14,
                fontweight='bold', color=c if v==best else GRAY)
        ax.text(0.5, yoff-0.16, m, transform=ax.transAxes,
                ha='center', fontsize=7.5, color=c)
    ax.set_title(label, fontsize=9.5, fontweight='bold', color=DARK)

ax2 = fig.add_subplot(gs[1, :])
ax2.set_facecolor(BG)
bar_data = [('Test Acc', [acc_a, acc_b]),
            ('Val Acc',  [val_acc_a, val_acc_b]),
            ('Test Loss',[loss_a, loss_b])]
x, w = np.arange(3), 0.3
for k, (mname, c) in enumerate(zip(model_names, colors_m)):
    vals = [d[1][k] for d in bar_data]
    bars = ax2.bar(x + k*w - w/2, vals, w, color=c, alpha=0.85, label=mname)
    for bar, v in zip(bars, vals):
        ax2.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
                 f'{v:.3f}', ha='center', va='bottom', fontsize=8.5,
                 fontweight='bold', color=c)
ax2.set_xticks(x)
ax2.set_xticklabels([d[0] for d in bar_data], fontsize=10)
ax2.set_ylim(0, 1.2)
ax2.legend(frameon=False, fontsize=9)
ax2.set_title('Metric Comparison', fontsize=11, fontweight='bold', color=DARK)
for sp in ax2.spines.values(): sp.set_visible(False)

delta = acc_b - acc_a
fig.text(0.5, -0.01, f'▲ Test accuracy improved by {delta:+.2%} ({delta*100:+.2f} pp)',
         ha='center', fontsize=10.5, fontweight='bold', color=TEAL)
fig.suptitle('Results Summary', fontsize=15, fontweight='bold',
             color=DARK, x=0.04, ha='left', y=1.02)
savefig('01_results_dashboard.png')
plt.show()
```




```
# ── D2. Training Curves ──────────────────────────────────────────────────
hist_a_dict = history_a.history
ft_epoch    = hist_b_combined['phase1_epochs']

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

def plot_curve(ax, hist, color, title, metric, ft_start=None):
    train = hist.get(metric, [])
    val   = hist.get('val_' + metric, [])
    ep    = np.arange(1, len(train)+1)
    ax.set_facecolor(BG)
    if len(train): ax.plot(ep, train, color=color, lw=2, label='Train')
    if len(val):   ax.plot(np.arange(1,len(val)+1), val, color=color,
                           lw=2, ls='--', alpha=0.75, label='Val')
    if ft_start:
        ax.axvline(ft_start, color=GRAY, lw=1, ls=':')
        ax.text(ft_start+0.3, ax.get_ylim()[0], 'fine-tune\nstart',
                fontsize=7.5, color=GRAY)
    ax.set_title(title, fontsize=10, fontweight='bold', color=DARK)
    ax.set_xlabel('Epoch', fontsize=9)
    ax.legend(fontsize=8, frameon=False)
    ax.tick_params(labelsize=8)
    for sp in ax.spines.values(): sp.set_visible(False)

plot_curve(axes[0,0], hist_a_dict,      BLUE, 'Accuracy — Baseline CNN', 'accuracy')
plot_curve(axes[0,1], hist_b_combined,  TEAL, 'Accuracy — Improved',     'accuracy', ft_start=ft_epoch)
plot_curve(axes[1,0], hist_a_dict,      BLUE, 'Loss — Baseline CNN',     'loss')
plot_curve(axes[1,1], hist_b_combined,  TEAL, 'Loss — Improved',         'loss',     ft_start=ft_epoch)

fig.suptitle('Training Curves', fontsize=15, fontweight='bold',
             color=DARK, x=0.04, ha='left', y=1.01)
plt.tight_layout()
savefig('02_training_curves.png')
plt.show()
```




```
# ── D3. Parameter Count vs Accuracy ──────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 5.5))
ax.set_facecolor(BG)
for sp in ax.spines.values(): sp.set_visible(False)

for name, p, a, c, ms in [
    ('Baseline CNN\n(from scratch)', params_a, acc_a, BLUE, 200),
    ('Improved\n(EfficientNetV2S)',  params_b, acc_b, TEAL, 260),
]:
    ax.scatter(p, a, s=ms, color=c, alpha=0.9, zorder=4, edgecolors='white', lw=1.5)
    ax.annotate(name, (p, a), xytext=(15 if p < (params_a+params_b)/2 else -15, 14),
                textcoords='offset points', fontsize=9, fontweight='bold', color=c,
                arrowprops=dict(arrowstyle='->', color=c, lw=1))

ax.annotate('', xy=(params_b, acc_b), xytext=(params_a, acc_a),
    arrowprops=dict(arrowstyle='->', color=GRAY, lw=1.5, linestyle='dashed'))
ax.text((params_a+params_b)/2+0.3, (acc_a+acc_b)/2+0.01,
        f'{acc_b-acc_a:+.2%} acc  /  {params_b/params_a:.1f}× params',
        ha='left', va='bottom', fontsize=8.5, color=GRAY, style='italic')

ax.set_xlabel('Total Parameters (millions)', fontsize=10)
ax.set_ylabel('Test Accuracy', fontsize=10)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v,_: f'{v:.0%}'))
pad = max(params_b-params_a, 1)*0.15
ax.set_xlim(max(0, params_a-pad*2), params_b+pad*3)
rng = abs(acc_b-acc_a)
ax.set_ylim(min(acc_a,acc_b)-rng*0.8, max(acc_a,acc_b)+rng*0.8)
ax.set_title('Parameter Count vs Test Accuracy', fontsize=13,
             fontweight='bold', color=DARK)
plt.tight_layout()
savefig('03_param_accuracy.png')
plt.show()
```



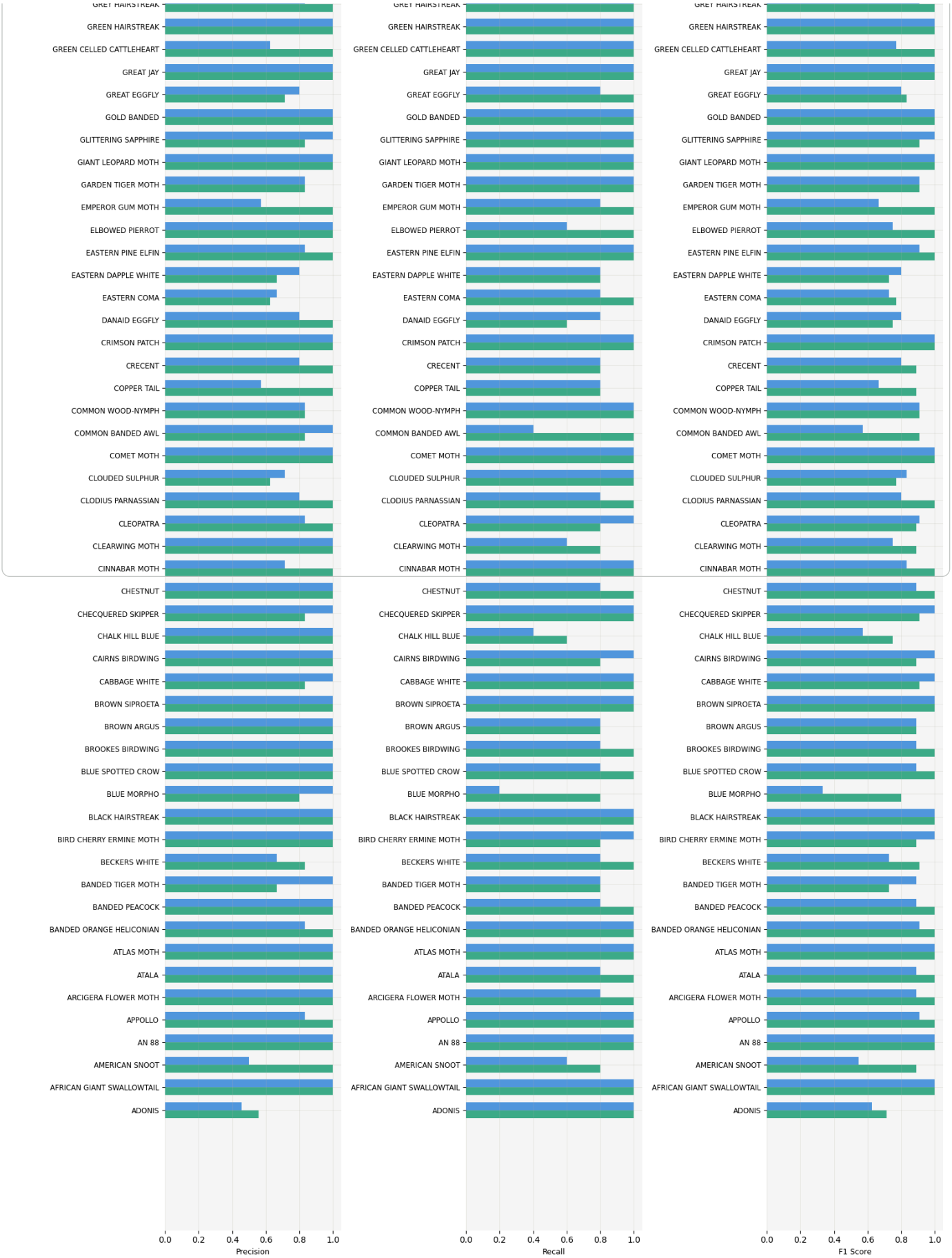

```
# ── D4. Collect predictions (used by all remaining cells) ──────────────────
from sklearn.metrics import classification_report, precision_recall_fscore_support

def get_preds(model, ds):
    y_true, y_pred = [], []
    for imgs, labels in ds:
        preds = model.predict(imgs, verbose=0)
        y_pred.extend(np.argmax(preds, axis=1))
        y_true.extend(labels.numpy())
    return np.array(y_true), np.array(y_pred)

print('Getting predictions — Baseline...')
yt_a, yp_a = get_preds(model_a, test_ds_a)
print('Getting predictions — Improved...')
yt_b, yp_b = get_preds(model_b, test_ds_b)

report_a = classification_report(yt_a, yp_a, target_names=class_names, output_dict=True, zero_division=0)
report_b = classification_report(yt_b, yp_b, target_names=class_names, output_dict=True, zero_division=0)
prec_a, rec_a, f1_a, _ = precision_recall_fscore_support(yt_a, yp_a, labels=range(num_classes), zero_division=0)
prec_b, rec_b, f1_b, _ = precision_recall_fscore_support(yt_b, yp_b, labels=range(num_classes), zero_division=0)
print('Done.')
```

```
Getting predictions — Baseline...
Getting predictions — Improved...
Done.
```

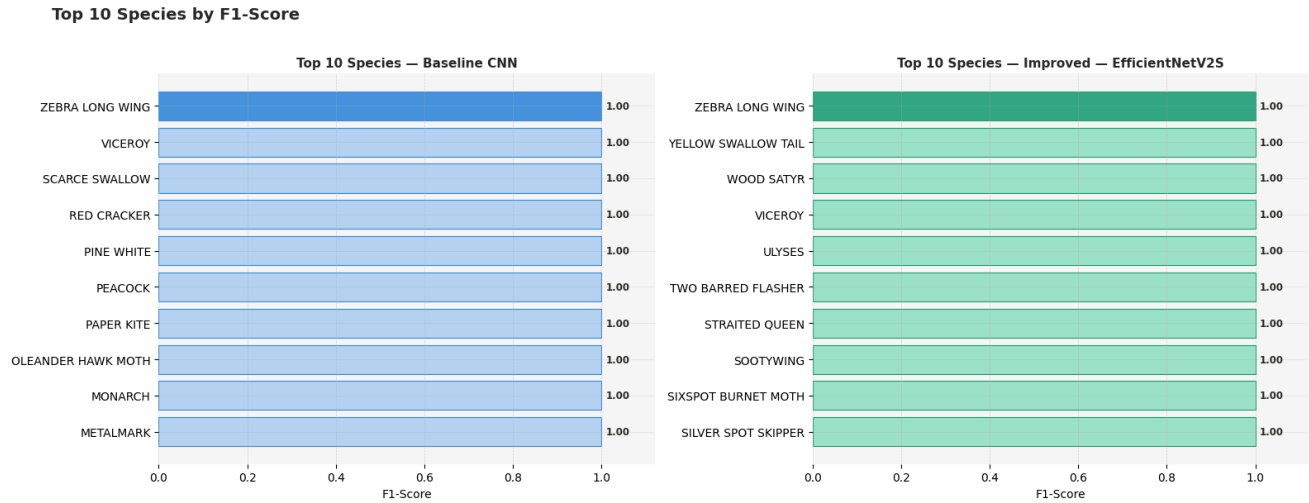
```
# ── D5. Per-class Precision / Recall / F1 ──────────────────────────────────
fig, axes = plt.subplots(1, 3, figsize=(15, max(4, num_classes*0.4+2)))
y_pos, h = np.arange(num_classes), 0.35

for ax, (metric, va, vb) in zip(axes, [('Precision',prec_a,prec_b),
                                        ('Recall',   rec_a, rec_b),
                                        ('F1 Score', f1_a,  f1_b)]):
    ax.set_facecolor(BG)
    ax.barh(y_pos+h/2, va, h, color=BLUE, alpha=0.85, label='Baseline')
    ax.barh(y_pos-h/2, vb, h, color=TEAL, alpha=0.85, label='Improved')
    ax.set_yticks(y_pos); ax.set_yticklabels(class_names, fontsize=8.5)
    ax.set_xlim(0, 1.05); ax.set_xlabel(metric, fontsize=9)
    ax.set_title(f'Per-class {metric}', fontsize=10, fontweight='bold', color=DARK)
    ax.legend(fontsize=8, frameon=False)
    for sp in ax.spines.values(): sp.set_visible(False)

fig.suptitle('Per-class Metrics — Baseline vs Improved', fontsize=14,
             fontweight='bold', color=DARK, x=0.04, ha='left', y=1.02)
plt.tight_layout()
```

```
savefig('04_per_class.png')
plt.show()
```

Per-class Metrics — Baseline vs Improved
Per-class Precision
Per-class Recall
Per-class F1 Score
Baseline
Improved
ZEBRA LONG WING
YELLOW SWALLOW TAIL
WOOD SATYR
WHITE LINED SPHINX MOTH
VICEROY
ULYSES
TWO BARRED FLASHER
TROPICAL LEAFWING
STRAITED QUEEN
SOUTHERN DOGFACE
SOOTYWING
SLEEPY ORANGE
SIXSPOT BURNET MOTH
SILVER SPOT SKIPPER
SCARCE SWALLOW
ROSY MAPLE MOTH
RED SPOTTED PURPLE
RED POSTMAN
RED CRACKER
RED ADMIRAL
QUESTION MARK
PURPLISH COPPER
PURPLE HAIRSTREAK
POPINJAY
POLYPHEMUS MOTH
PIPEVINE SWALLOW
PINE WHITE
PEACOCK
PAPER KITE
PAINTED LADY
ORCHARD SWALLOW
ORANGE TIP
ORANGE OAKLEAF
OLEANDER HAWK MOTH
MOURNING CLOAK
MONARCH
MILBERTS TORTOISESHELL
METALMARK
MESTRA
MANGROVE SKIPPER
MALACHITE
MADAGASCAN SUNSET MOTH
LUNA MOTH
LARGE MARBLE
JULIA
Iphiclus sister
IO MOTH
INDRA SWALLOW
HUMMING BIRD HAWK MOTH
HERCULES MOTH


GREY HAIRSTREAK
GREEN HAIRSTREAK
GREEN CELLED CATTLEHEART
GREAT JAY
GREAT EGGFLY
GOLD BANDED
GLITTERING SAPPHIRE
GIANT LEOPARD MOTH
GARDEN TIGER MOTH
EMPEROR GUM MOTH
ELBOWED PIERROT
EASTERN PINE ELFIN
EASTERN DAPPLE WHITE
EASTERN COMA
DANAID EGGFLY
CRIMSON PATCH
CRECENT
COPPER TAIL
COMMON WOOD-NYMPH
COMMON BANDED AWL
COMET MOTH
CLOUDED SULPHUR
CLODIUS PARNASSIAN
CLEOPATRA
CLEARWING MOTH
CINNABAR MOTH
CHESTNUT
CHECQUERED SKIPPER
CHALK HILL BLUE
CAIRNS BIRDWING
CABBAGE WHITE
BROWN SIPROETA
BROWN ARGUS
BROOKES BIRDWING
BLUE SPOTTED CROW
BLUE MORPHO
BLACK HAIRSTREAK
BIRD CHERRY ERMINE MOTH
BECKERS WHITE
BANDED TIGER MOTH
BANDED PEACOCK
BANDED ORANGE HELICONIAN
ATLAS MOTH
ATALA
ARCIGERA FLOWER MOTH
APPOLLO
AN 88
AMERICAN SNOOT
AFRICAN GIANT SWALLOWTAIL
ADONIS
0.0 0.2 0.4 0.6 0.8 1.0
Precision
Recall
F1 Score

```
# ── D6. Top 10 Species by F1-Score ──────────────────────────────────────────
def top10_f1(report, names):
    perf = [(s, report[s]['f1-score']) for s in names if s in report]
    perf.sort(key=lambda x: x[1])
    return perf[-10:]

best_a = top10_f1(report_a, class_names)
best_b = top10_f1(report_b, class_names)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for ax, best, color, lcolor, label in [
    (axes[0], best_a, BLUE, BLUE_L, 'Baseline CNN'),
    (axes[1], best_b, TEAL, TEAL_L, 'Improved — EfficientNetV2S'),
]:
    names_p  = [x[0] for x in best]
    scores_p = [x[1] for x in best]
    ax.set_facecolor(BG)
    bars = ax.barh(names_p, scores_p, color=lcolor, edgecolor=color, linewidth=0.8)
    bars[-1].set_facecolor(color); bars[-1].set_alpha(0.9)
    for bar in bars:
        w = bar.get_width()
        ax.text(w+0.01, bar.get_y()+bar.get_height()/2,
                f'{w:.2f}', va='center', fontsize=8.5, color=DARK, fontweight='bold')
    ax.set_xlabel('F1-Score', fontsize=10); ax.set_xlim(0, 1.12)
    ax.set_title(f'Top 10 Species — {label}', fontsize=11, fontweight='bold', color=DARK)
    ax.grid(axis='x', linestyle='--', alpha=0.5)
    for sp in ax.spines.values(): sp.set_visible(False)

fig.suptitle('Top 10 Species by F1-Score', fontsize=14,
             fontweight='bold', color=DARK, x=0.04, ha='left', y=1.02)
plt.tight_layout()
savefig('05_top10_species.png')
plt.show()
```




```
# ── D7. Top 10 Most Common Mix-Ups ──────────────────────────────────────────
def top10_mixups(y_true, y_pred, names, n=10):
    df = pd.DataFrame({'Actual': y_true, 'Predicted': y_pred})
    df = df[df['Actual'] != df['Predicted']]
    df['Actual']    = df['Actual'].map(lambda x: names[x])
    df['Predicted'] = df['Predicted'].map(lambda x: names[x])
    top    = df.value_counts().head(n)
    labels = [f"True: {a}\nPred: {p}" for a, p in top.index]
    return labels, top.values

labels_a, counts_a = top10_mixups(yt_a, yp_a, class_names)
labels_b, counts_b = top10_mixups(yt_b, yp_b, class_names)

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
```